

Relay — API Specification

Status: Draft for build · **Owner:** Systems/Backend · **Last updated:** June 2026

This is the contract between backend, frontend, and the agent worker. **Freeze this before parallel work begins.** All bodies are JSON. Base URL `/(api)/v1`. Auth omitted for the hackathon (single-org assumption); add a bearer token before any real deployment.

Conventions

- Timestamps: ISO-8601 UTC.
 - IDs: prefixed UUIDs (doc_..., ses_..., card_..., lead_...).
 - Errors: `{ "error": { "code": string, "message": string } }` with appropriate HTTP status.
 - mode enum: `live | desk | intake`.
-

REST endpoints

Ingestion

POST `/documents` — upload & ingest a document

- Body (multipart): `file` (PDF/docx/txt), `title?`, `tags?[]`
- Flow: Unsiloed parse → chunk → embed → Moss index + Postgres.
- 202 → `{ "document_id": "doc_...", "status": "processing" }`

GET `/documents` → `{ "documents": [ { document_id, title, status, chunk_count, created_at } ] }`

GET `/documents/{document_id}` → full record incl. status (processing|ready|failed).

DELETE `/documents/{document_id}` → 204. Removes chunks from Moss + Postgres.

Sessions

POST `/sessions` — start a session/call

- Body: `{ "mode": "live", "livekit_room?": string, "customer_id?": string }`
- 201 → `{ "session_id": "ses_...", "ws_url": "/ws/sessions/ses_...", "livekit_token?": string }`

GET `/sessions/{session_id}` → `{ session_id, mode, status,`

started_at, ended_at, card_count }

POST /sessions/{session_id}/end → 200 → { status: "ended" }

GET /sessions/{session_id}/cards → { "cards": [Card] } (replay)

GET /sessions/{session_id}/transcript → { "utterances": [Utterance] }

Query (manual fallback / Desk)

POST /query

- Body: { "session_id?": string, "mode": "desk", "text": string, "customer_id?": string }
- 200 → { "card": Card | null } (null = no relevant grounding)

Leads (Intake mode)

GET /leads → { "leads": [Lead] }

GET /leads/{lead_id} → Lead
POST /leads/{lead_id}/route → { "routed_to": string } (e.g. Slack ping)

WebSocket — /ws/sessions/{session_id}

Bidirectional event stream for live transcript + cards. Envelope:

```
{ "type": "string", "ts": "ISO-8601", "data": { } }
```

Server → client

type	data
transcript.partial	{ speaker, text }
transcript.final	{ utterance_id, speaker, text }
card.new	Card
card.update	{ card_id, ...partial } (<i>for streamed synthesis</i>)
session.status	{ status: "active" "ended", retrieval_backend: "moss" "pgvector" }
error	{ code, message }

Client → server

type	data
mode.set	{ mode }

type	data
query.manual	{ text }
card.pin	{ card_id }
card.dismiss	{ card_id }

Shared object shapes

```
// Card
{
  "card_id": "card_...",
  "session_id": "ses_...",
  "mode": "live",
  "answer": "Our uptime SLA is 99.9% per the MSA.",
  "sources": [ { "document_id": "doc_...", "title": "MSA", "snippet": "...",
"score": 0.91 } ],
  "trigger_text": "what's your uptime SLA?",
  "latency_ms": 420,
  "created_at": "2026-06-06T18:04:11Z"
}

// Utterance
{ "utterance_id": "utt_...", "session_id": "ses_...", "speaker": "prospect", "text":
 "...", "ts": "..." }

// Lead (Intake)
{
  "lead_id": "lead_...",
  "session_id": "ses_...",
  "name": "...", "company": "...", "email": "...",
  "qualifiers": { "budget": "...", "timeline": "...", "need": "..." },
  "score": 82,
  "status": "hot",           // hot | warm | cold
  "routed_to": "#sales",
  "created_at": "..."
}
```

Error codes

document_unsupported · document_too_large · session_not_found ·
retrieval_unavailable · no_grounding (used as card: null, not an error) ·
internal_error.

Related docs

DATA_MODEL.md · TECHNICAL_DESIGN.md